

CÓDIGO CONCURRENTE CON VIBECODING: MÉTRICAS Y ANÁLISIS

Integrantes:

Rodrigo Ariel LA GIGLIA

Rlagiglia248@alumno.unlam.edu.ar

Martin Ricardo NOVOA ROMERO

mnovoaromero@alumno.unlam.edu.ar

Agustin Elias GONZALEZ

agustinegonzalez@alumno.unlam.edu.ar

Asignatura:

(3658) PROGRAMACION CONCURRENTE

Fecha - Versión:

08/10/2025 – Versión 1.01

Resumen:

Este trabajo de investigación explora la generación de código concurrente mediante Vibecoding, específicamente para el problema de lectores y escritores. Se utilizó un prompt técnico diseñado según los lineamientos de "Análisis preliminar de herramientas para Vibecoding aplicadas al desarrollo de software"[1] con el fin de evaluar el desempeño de la herramienta Copilot integrada en Visual Studio para la construcción de soluciones que requieren sincronización. El análisis posterior se centró en la aplicación de métricas de software, particularmente la complejidad ciclomatica, para estimar la calidad estructural y la mantenibilidad del código generado. Los resultados indican que la implementación satisface los requisitos propuestos, aunque se identificaron oportunidades de mejora relacionadas con el uso de variables globales y la posibilidad de inanición de procesos. Este ejercicio contribuyó al entendimiento práctico de la concurrencia y permitió reflexionar sobre el aporte de las nuevas tecnologías en el desarrollo de software.

Palabras clave: vibecoding, concurrencia, lectores-escritores, métricas de software

I. CONTEXTO

El presente trabajo se ubica en el área del conocimiento de la programación concurrente, dentro de la temática de sistemas operativos y sincronización de procesos. La actividad se desarrolla en el marco académico de la Universidad Nacional de La Matanza (UNLaM), en la asignatura Programación Concurrente.

II. INTRODUCCIÓN

El auge de herramientas de inteligencia artificial aplicada a la programación ha impulsado investigaciones que buscan establecer criterios objetivos para evaluar la calidad del código generado. En el marco de Vibecoding, se destacan dos dimensiones:

- **Eficiencia funcional:** si el programa cumple los requisitos de sincronización.
- **Calidad estructural:** si el código es comprensible, mantenible y libre de riesgos de complejidad excesiva.

Este trabajo aborda ambas dimensiones mediante la construcción de un *prompt técnico* que describe el problema de lectores y escritores, y el análisis del código resultante con la métrica de complejidad ciclomatica (CC) propuesta por McCabe [2].

III. MÉTODOS

3.1 Generación de código

- Herramienta de Vibecoding: Visual Studio con GitHub Copilot.
- Prompt técnico: diseñado según lineamientos de "Análisis preliminar de herramientas para Vibecoding aplicadas al desarrollo de software" [1], describiendo el problema de lectores y escritores.
- Requisitos incluidos: parametrización de lectores y escritores, accesos temporizados, exclusión mutua para escritores, concurrencia para lectores y resumen final de modificaciones.

Prompt Técnico – Lectores y Escritores con Ejecución Temporizada

- LENGUAJE DE PROGRAMACIÓN: Python
- PLATAFORMA: Windows
- FUNCIONALIDAD A IMPLEMENTAR:

Desarrollar un sistema concurrente que simule el acceso a un documento compartido por parte de múltiples procesos, de acuerdo a los requisitos y restricciones específicas. El contexto de la aplicación es que varios usuarios acceden a un mismo documento:

- Los lectores representan a los usuarios que abren el documento solo para consultar la información.

- Los escritores representan a los editores o administradores que pueden modificar el contenido (por ejemplo, actualizar datos o corregir errores).

En este escenario, muchos usuarios pueden leer el documento al mismo tiempo sin problemas, pero cuando un editor necesita actualizarlo, debe hacerlo de manera exclusiva, evitando que alguien más lo lea o lo modifique durante ese instante.

- **REQUISITOS ESPECÍFICOS**

- La cantidad de lectores y escritores debe recibirse como parámetro en el código.
- Cada lector/escritor se ejecutará durante un tiempo fijo X segundos (definido en una constante), generando accesos concurrentes aleatorios (entre 3 y 10 accesos cada uno) mientras el programa está activo.
- El programa principal debe esperar a que todos los hilos terminen su ejecución antes de finalizar.
- El documento compartido se representará como una variable en memoria.
- Los lectores mostrarán el contenido actual del documento cuando acceden, indicando “lector número N lee X”.
- Los escritores modificarán el contenido escribiendo su número de escritor, mostrando por consola “Escritor número N escribiendo X”.
- Al terminar la simulación, el programa imprimirá un resumen con el historial de cambios realizados por los escritores.
- La sincronización debe garantizar que:
 - Varios lectores pueden acceder al mismo tiempo.
 - Un escritor requiere acceso exclusivo.

RESTRICCIONES:

No puede haber condiciones de carrera: lectores y escritores deben usar mecanismos de sincronización adecuados.

El programa principal no finaliza hasta que todos los lectores y escritores hayan terminado su ejecución.

3.2 Selección de la métrica

Se eligió la complejidad ciclomatica (CC) por su relevancia en la medición de la complejidad de flujo de control y su impacto en la mantenibilidad [2].

3.3 Procedimiento

- Generación automática del código en Visual Studio Code con la herramienta GitHub Copilot que utiliza el modelo de lenguaje Claude Sonnet 3.5. (ver código en Anexo A)
- Validación funcional mediante pruebas de ejecución.
- Análisis estructural con la herramienta radon (Python).

- Registro de valores de CC por función y cálculo del promedio global.
- Interpretación de resultados en relación con los umbrales propuestos en la literatura.

IV. RESULTADOS Y OBJETIVOS

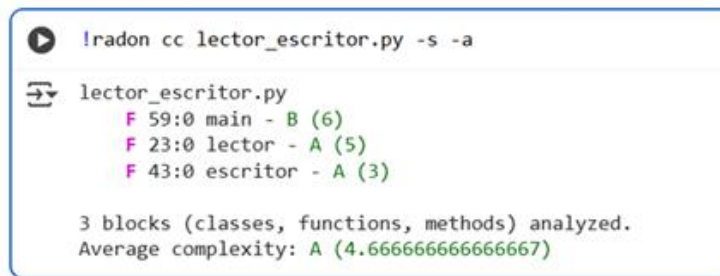
4.1 Correctitud funcional

El programa cumple los requisitos planteados:

- Accesos concurrentes múltiples para lectores.
- Exclusión mutua para escritores.
- Accesos temporizados aleatorios.
- Resumen final con historial de cambios.

4.2 Complejidad ciclomatica

Se utilizó la herramienta Radon [3] para calcular la complejidad estructural del código. Los resultados indican que el código mantiene una baja complejidad en todas sus funciones, lo cual es favorable para la calidad y el mantenimiento:



```
!radon cc lector_escritor.py -s -a

lector_escritor.py
  F 59:0 main - B (6)
  F 23:0 lector - A (5)
  F 43:0 escritor - A (3)

3 blocks (classes, functions, methods) analyzed.
Average complexity: A (4.666666666666667)
```

Fig. 1. Complejidad ciclomática del código lector_escritor.py. Se muestra la calificación obtenida por cada función (main, lector, escritor) y el promedio general (A).

Según los criterios de McCabe [2], los valores obtenidos (CC entre 4 y 6) caen en el rango de baja complejidad ($CC \leq 10$). A pesar de que main() alcanza un valor de 6, se mantiene dentro del umbral ideal. El valor de $CC=6$ en main() se explica por la presencia de múltiples puntos de decisión:

- Bucle for para la creación de hilos lectores.
- Bucle for para la creación de hilos escritores.
- Bucle for para iniciar (t.start()) los hilos.
- Bucle for para esperar (t.join()) a que los hilos terminen.

Estos cuatro bucles, más el punto de entrada y salida de la función, justifican el valor de $CC=6$.

4.2 Limitaciones del Diseño de Concurrencia

- La solución implementada utiliza variables globales, que son funcionales para el paso de memoria compartida, pero sacrifican el encapsulamiento. Refactorizar la estructura en una clase permitiría agrupar el recurso (documento) junto con sus mecanismos de sincronización (lectores_lock, escritor_lock), reduciendo el acoplamiento global y mejorando significativamente la mantenibilidad futura del código [4], [5].
- Riesgo de Inanición (Starvation) en el Patrón Lectores-Escritores: El código implementa la solución del patrón Lectores-Escritores con prioridad a los lectores. Esta implementación, al favorecer incondicionalmente a los lectores una vez que el primer lector adquiere acceso, puede provocar que los escritores sufran inanición (starvation), siendo impedidos indefinidamente de ganar acceso al recurso mientras haya un flujo constante de lectores [6].
- Lock Muerto Persistente (Dead Code): El lock mutex se declara en el código (mutex = threading.Lock()) pero nunca se utiliza. Su existencia aumenta la complejidad percibida, introduce costos de mantenimiento innecesarios y viola estándares de calidad, ya que sugiere una funcionalidad que no está presente [7].
- Inconsistencia en la Gestión de Locks: Si bien la función lector() utiliza correctamente el gestor de contexto (with), la función escritor() utiliza la técnica acquire() y release() de forma insegura, sin un bloque try...finally que garantice la liberación ante una excepción. La liberación de locks debe garantizarse para evitar fugas de recursos y bloqueos mutuos (deadlock) [8].

4.3 Desviación de Requerimientos en Iteraciones con IA

A través de las sucesivas correcciones realizadas con ayuda de la inteligencia artificial, se obtienen mejoras en la claridad y organización del código. Sin embargo, la IA puede dejar de atender ciertos requerimientos definidos en el primer prompt, lo que obliga a revisar manualmente cada versión para mantener la coherencia con los objetivos iniciales.

V. CONCLUSIONES

La investigación evidencia que el uso de métricas como la complejidad ciclomatica proporciona un criterio objetivo de evaluación que la IA supera satisfactoriamente. Sin embargo, este ejercicio también subraya la necesidad de complementar las métricas de complejidad con una verificación profunda de los patrones de diseño y las prácticas de seguridad específicas de la concurrencia. El código generado automáticamente cumple el requisito funcional, pero requiere refactorización manual para corregir las vulnerabilidades de diseño y alcanzar la calidad estructural necesaria para la mantenibilidad a largo plazo.

VI. ANEXO A. CÓDIGO GENERADO

```
import threading
import time
import random

# Constantes de configuración
NUM_LECTORES = 4 # Cambiar según necesidad
NUM_ESCRITORES = 2 # Cambiar según necesidad
TIEMPO_EJECUCION = 8 # Segundos que cada hilo estará activo
MIN_ACCESOS = 3
MAX_ACCESOS = 10

# Documento compartido y su historial
documento = {"contenido": '"Documento inicial"'}
historial_cambios = []
historial_lock = threading.Lock()

# Sincronización lectores-escriptores
mutex = threading.Lock()
lectores = 0
lectores_lock = threading.Lock()
escriptor_lock = threading.Lock()

def lector(id_lector):
    global lectores
    accesos = random.randint(MIN_ACCESOS, MAX_ACCESOS)
    tiempo_inicio = time.time()
    for i in range(accesos):
        if time.time() - tiempo_inicio > TIEMPO_EJECUCION:
            break
        time.sleep(random.uniform(0.2, 1.2)) # Simula acceso aleatorio

        with lectores_lock:
            lectores += 1
            if lectores == 1:
                escritor_lock.acquire()
        # Leer documento
        print(f"Lector numero {id_lector} lee {documento['contenido']}")
        with lectores_lock:
            lectores -= 1
            if lectores == 0:
                escritor_lock.release()

def escritor(id_escritor):
    accesos = random.randint(MIN_ACCESOS, MAX_ACCESOS)
    tiempo_inicio = time.time()
    for i in range(accesos):
```

```

    if time.time() - tiempo_inicio > TIEMPO_EJECUCION:
        break
    time.sleep(random.uniform(0.5, 1.5)) # Simula acceso aleatorio

    escritor_lock.acquire()
    nuevo_contenido = f"Escrito por escritor {id_escritor} acceso {i+1}"
    documento['contenido'] = nuevo_contenido
    print(f"Escritor numero {id_escritor} escribiendo {nuevo_contenido}")
    with historial_lock:
        historial_cambios.append((id_escritor, nuevo_contenido))
    escritor_lock.release()

def main():
    hilos = []

    # Crear hilos lectores
    for i in range(1, NUM_LECTORES + 1):
        t = threading.Thread(target=lector, args=(i,))
        hilos.append(t)

    # Crear hilos escritores
    for i in range(1, NUM_ESCRITORES + 1):
        t = threading.Thread(target=escritor, args=(i,))
        hilos.append(t)

    # Iniciar todos los hilos
    for t in hilos:
        t.start()

    # Esperar a que todos terminen
    for t in hilos:
        t.join()

    # Mostrar resumen de cambios
    print("\nResumen de cambios realizados por los escritores:")
    for idx, (id_escritor, contenido) in enumerate(historial_cambios, 1):
        print(f"{idx}. Escritor {id_escritor}: {contenido}")

if __name__ == "__main__":
    main()

```


VII. REFERENCIAS Y BIBLIOGRAFÍA

- [1] E. Carnuccio, M. Volker, C. Tomalino, M. Adagio, D. Hirschfeldt, y A. Vera, "Análisis preliminar de herramientas para Vibecoding aplicadas al desarrollo de software," CONAIIISI 2025.
- [2] T. J. McCabe, "A complexity measure," *IEEE Transactions on Software Engineering*, vol. SE-2, no. 4, pp. 308–320, Dec. 1976.
- [3] "Radon Documentation - Python Code Metrics." Accessed: Sept. 29, 2025. [Online].
- [4] R. S. Pressman y B. R. Maxim, *Ingeniería del Software: Un Enfoque Práctico*. McGraw-Hill, 9ª ed., 2021.
- [5] E. Gamma, R. Helm, R. Johnson, y J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995.
- [6] A. Silberschatz, P. B. Galvin, y G. Gagne, *Fundamentos de Sistemas Operativos*. Addison-Wesley, 10ª ed., 2021, pp. 278-280.
- [7] M. Fowler, *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999, p. 77.
- [8] B. Goetz, T. Peierls, J. Bloch, J. Bowbeer, D. Holmes, y C. Lea, *Java Concurrency in Practice*. Addison-Wesley, 2006, pp. 119-120.